Teaching Proof in the Age of Verification: Lessons from the Cathedral for Mathematics Education

Jason Robert Gochanour

April 20, 2026

Abstract

The Cathedral—a machine-verified reduction of the Riemann Hypothesis in Lean 4—was built in 23 days by a computer scientist (not a professional mathematician) working with AI assistance. This paper examines what this implies for mathematics education. We identify five pedagogical shifts: (1) proof as *compilation* rather than persuasion, (2) error as *data* rather than failure, (3) axioms as *design choices* rather than sacred truths, (4) collaboration with AI as a *cognitive amplifier* rather than a crutch, and (5) archival as *curriculum* rather than waste. We propose concrete changes to undergraduate and graduate mathematics curricula, argue that formal verification should be taught alongside traditional proof writing, and suggest that the Cathedral's methodology—build, break, archive, audit— is a teachable skill that transcends mathematics.

Contents

1 The Uncomfortable Question

A computer scientist with no PhD in mathematics, working with AI assistance, produced a 78-file machine-verified proof system that reduces one of the most famous open problems in mathematics to two axioms. He did this in 23 days.

This raises questions that mathematics educators must confront:

1. If the tools for formal verification are this accessible, should we be teaching proof differently?
2. If AI can generate 85% of proof code, what should humans learn?
3. If machine verification can catch errors that experts miss (the High-Frequency Trap, the False Dedekind Reciprocity), what is the role of mathematical intuition?
4. If 96 archived files (failed attempts) are essential to the discovery process, should we teach failure as curriculum?

This paper does not claim that mathematicians are obsolete. The Cathedral required deep mathematical taste, architectural vision, and creative problem-solving that no AI currently possesses. But the *mechanics* of proof—the line-by-line construction of logical arguments—have been fundamentally changed by verification technology. Education must adapt.

2 Five Pedagogical Shifts

2.1 Shift 1: Proof as Compilation

Traditional model: A proof is a persuasive argument written for a human reader. Its correctness is judged by whether experts find it convincing.

Cathedral model: A proof is a program that compiles. Its correctness is judged by whether the type checker accepts it. The audience is the compiler, not a professor.

Pedagogical implication: Students can get *immediate, objective feedback* on whether their proof is correct. No waiting for grading. No ambiguity about whether a step is justified. The compiler either accepts it or it doesn't.

This changes the emotional experience of learning proof:

	Traditional	**Verified**
Feedback speed	Days (grading)	Seconds (compilation)
Feedback objectivity	Subjective	Objective
Error granularity	"This step is wrong"	Exact error location
Partial credit	Yes	No (but partial progress visible)
Student anxiety	High (uncertainty)	Lower (clarity)

Concrete proposal: Introduce Lean 4 proof exercises alongside traditional proof writing in discrete mathematics and introduction to proof courses. Students write both a human-readable proof and a machine-checked proof, learning that these are complementary skills.

2.2 Shift 2: Error as Data

Traditional model: Errors in proofs are failures. A wrong proof is a bad grade.

Cathedral model: Errors are *discoveries*. The Cathedral's five traps—the High-Frequency Trap, the False Dedekind Reciprocity, the Triangle Inequality Trap, the Hyperplane Trap, and the Ghost Axiom Phenomenon—were each detected by the machine and each redirected the proof toward a better path.

The 96 archived files are not failures. They are the empirical record of a systematic exploration of mathematical space. The False Dedekind Reciprocity, detected numerically at $(a, b) = (3, 2)$ before any proof attempt, saved potentially weeks of wasted effort and motivated the Parseval Bridge—the key innovation.

Pedagogical implication: Students should be graded not just on correct proofs but on their *proof journals*: the record of what they tried, what failed, and what they learned from failures. The archive is as valuable as the proof.

Concrete proposal: In proof-based courses, require students to submit a "proof diary" alongside each assignment: what approaches they tried, why each failed, and how failures informed their final approach. Grade the diary.

2.3 Shift 3: Axioms as Design Choices

Traditional model: Axioms are foundational truths that mathematicians agree on. Students learn to prove things *from* axioms but rarely think about the axioms themselves.

Cathedral model: Axioms are *engineering decisions*. The Cathedral started with 56 axioms and reduced to 42 (7 on the crown path) through systematic campaigns. Each axiom was named, classified by tier, tracked by dependents, and evaluated for elimination. This is axiom *management*, not axiom worship.

Pedagogical implication: Students should learn to:

1. Identify the axioms in a proof (what are we assuming?).
2. Classify axioms by strength (how much does this assumption buy us?).
3. Ask whether axioms can be eliminated (can we prove this from something weaker?).
4. Understand the trade-off between axiom count and proof complexity.

Concrete proposal: Add "axiom awareness" to proof courses. For each proof, students must list the axioms used and classify them as "proved in Mathlib," "standard but unformalized," or "genuinely new assumption."

2.4 Shift 4: AI as Cognitive Amplifier

Traditional model: Students must produce proofs independently. Using tools beyond pen and paper is cheating.

Cathedral model: AI generated 85% of code by volume. The human provided 100% of the architectural vision. The result was a system that neither could have produced alone.

Pedagogical implication: We need to teach students how to *collaborate with AI* on mathematical reasoning:

- How to decompose a problem into tasks suitable for AI assistance.
- How to evaluate AI-generated proofs (the compiler helps, but understanding the proof is still the human's job).
- How to provide architectural direction that AI cannot generate on its own.
- When to trust AI output and when to verify independently.

The analogy: we teach students to use calculators without considering it cheating. We should teach them to use proof assistants and AI similarly—as tools that handle mechanical computation so the human can focus on creative reasoning.

Concrete proposal: Develop "AI-assisted proof" exercises where students are given a theorem and must:

1. Formulate a proof strategy (human).
2. Use AI to generate candidate proof steps (AI).
3. Verify the result in Lean 4 (compiler).
4. Write a human-readable explanation (human).

2.5 Shift 5: Archive as Curriculum

Traditional model: Published proofs are clean, polished artifacts. The messy process of discovery is hidden.

Cathedral model: The archive (96 files) is preserved alongside the active codebase (84 files). Failed approaches are documented with explanations of *why* they failed. The archive is a teaching resource.

Pedagogical implication: Students should study *failed proofs* as seriously as successful ones. Understanding why an approach fails develops mathematical maturity faster than reading correct proofs.

Concrete proposal: In graduate courses on analytic number theory or proof engineering, assign the Cathedral's archive as reading material. Students analyze a failed approach (e.g., the Vasyunin cotangent path) and write a report on:

1. What the approach attempted.
2. Where and why it failed.
3. What was learned from the failure.
4. How the failure informed the eventual solution.

3 Curriculum Recommendations

3.1 Undergraduate

1. **Introduction to Proof (Year 1–2):** Add Lean 4 exercises alongside traditional proofs. Start with propositional logic and basic set theory in Lean.

2. **Discrete Mathematics (Year 1–2):** Use Lean 4 for induction proofs, divisibility, and modular arithmetic. Students see that the computer checks their induction step the same way a professor would.

3. **Linear Algebra (Year 2):** Formalize basic matrix theorems (determinant properties, eigenvalue bounds). The Cathedral's Sherman–Morrison and Schur Complement proofs are accessible at this level.

4. **Analysis (Year 2–3):** Use Lean 4 to formalize ε-δ proofs. The precision required by the compiler teaches students to be exact about quantifiers.

3.2 Graduate

1. **Formal Methods seminar:** Study the Cathedral as a case study in large-scale formalization. Topics: axiom management, proof architecture, the archive as documentation.

2. **Analytic Number Theory:** Use the Cathedral's proof tree as a roadmap. Students attempt to formalize one of the 38 non-critical axioms as a class project.

3. **AI-Assisted Mathematics:** Study the human-AI collaboration model. Compare with AlphaProof, LeanDojo, and other AI-for-math systems. Discuss the "cognitive amplifier" vs "replacement" debate.

4 What Students Should Learn That Won't Change

Machine verification does not replace mathematical understanding. The Cathedral required:

- **Taste**: Recognizing that the Parseval Bridge was more promising than the Vasyunin path. No AI suggested this.

- **Intuition**: Sensing that the Rank-1 Mellin Miracle "should" work before proving it formally.

- **Architecture**: Designing the 11-module structure that made the proof manageable. This is software engineering applied to mathematics.

- **Persistence**: 96 failed files. Three weeks of 12-hour days. The machine checks the proof; the human must keep going when everything is failing.

- **Communication**: 17 papers for 17 audiences. The ability to explain the same idea to a physicist, a philosopher, and a senator. No compiler checks this.

These are human skills. They should remain at the center of mathematics education. What changes is the *mechanics*: the laborious, error-prone process of checking logical steps by hand is increasingly delegated to machines, freeing human minds for the creative work that machines cannot yet do.

5 Conclusion: The Compiler as Teacher

The Lean compiler is, in a sense, the most patient and honest mathematics teacher imaginable. It never gets tired. It never gives partial credit for a wrong proof. It never misses an error. It gives feedback in seconds. And it treats every student exactly the same way, regardless of background, reputation, or pedigree.

A computer scientist in New Mexico, with no doctorate in mathematics, working with AI tools available to anyone, built a 78-file formal proof system that addresses the most famous open problem in mathematics. He did this because the tools are now good enough that mathematical *taste* and *persistence* matter more than mathematical *pedigree*.

If we teach the next generation to use these tools—not as replacements for thinking, but as amplifiers for it—the next cathedral will be built by a student who hasn't been born yet.

> *The compiler does not care who you are.*
> *It only cares whether you are right.*
> *That is the most democratic thing*
> *in all of mathematics.*

References

[1] L. de Moura *et al.*, *The Lean 4 Theorem Prover and Programming Language*, CADE-28, 2021.

[2] The Mathlib Community, *Mathlib: A unified library of mathematics formalized in Lean 4*, 2024.

[3] J. Avigad, *Mathematics in Lean*, online textbook, 2023.

[4] K. Buzzard, *The Future of Mathematics*, ICM 2022 invited lecture.

[5] T. Hales, *Formal Abstracts*, Fields Institute, 2019.